Auditing the Effort Axis: A Cost-Proxy Check for Sampling-Based Performance Prediction of Configurable Systems

Anonymous Author(s)
Submission to ICSE 2027 – Research Track

***Abstract*—Sampling-based performance prediction for configurable systems is evaluated by accuracy and sample size, treating the number of measured configurations as a proxy for acquisition cost. When is that proxy safe, and how should authors check? We answer both. We define measured training cost—the sum of the measured costs of the configurations a strategy selects—and distill a lightweight, three-step *Cost-Proxy Audit*: inspect per-configuration cost spread, check per-strategy budget agreement, and recompute the decision under measured cost.**

We validate the audit across six evidence families—two replications of published configurable-system sampling settings and four extensions covering performance–energy trade-offs, workload sensitivity, hyperparameter learning curves, and Linux kernel releases. Sample size orders measured cost everywhere, so the proxy is not arbitrary; but the audit flags real problems: same-size samples can spend very different budgets, a larger sample can be cheaper than a smaller one, and replacing count by measured cost changes the recommended strategy–budget combination in 538 of $1{,}666$ decision contexts. The audit is cheap and reusable: it certifies when sample size is a safe budget and, when it is not, states which conclusion is count-based and which is measured-cost based.

***Index Terms*—configurable systems, performance prediction, sampling, measurement cost, empirical software engineering**

I. Introduction

Performance prediction for configurable software systems is usually framed as a sampling problem: measure a small set of configurations, train a model, and use it to predict the rest of the configuration space [1]–[5]. Because measurements are expensive, sampling studies compare strategies on accuracy and "effort," and in this literature effort almost always means the number of measured configurations. A strategy that measures 64 configurations is treated as twice as expensive as one that measures 32, and two strategies that both measure 64 configurations are treated as equally expensive.

That convention is useful, but it is also a hypothesis. It assumes that each configuration costs roughly the same to measure, or at least that sampling strategies do not systematically choose more expensive or cheaper regions of the configuration space. Configurable systems give many reasons to doubt this. In a video encoder, compression options can multiply encoding time; in an archiver, compression settings can change energy by orders of magnitude; in a compiler, optimization passes can turn a short compilation into a long one. In such settings, sample size remains a count, but it may no longer be a budget.

The gap is not the absence of a better sampler; it is the absence of a validity check for the effort axis itself. Prior work has optimized sampling strategies, compared learners, and reported accuracy–effort trade-offs, but effort is almost always operationalized as a configuration count. That convention is convenient and often reasonable, yet it silently turns every cost-effectiveness statement into a *construct-validity* claim: one count unit must stand for one comparable unit of acquisition cost. We test that assumption directly and treat its success or failure as the object of study, not as a means to a new sampler.

The issue is not merely accounting. Figure 1 gives the smallest version of the phenomenon on a real subject: lrzip in the performance–energy corpus. The goal is to predict energy, but the acquisition budget is the measured performance cost of the configurations selected for training. All strategies measure the same 16 configurations, so count-based effort assigns them the same cost. Measured cost does not: pilot-cost sampling spends 0.034% of the full measurement budget while diversity sampling spends 0.567%. Their errors are almost indistinguishable (51.3% vs. 52.4% MRE), yet their measured costs differ by $16.5\times$. The absolute error is high because this is a deliberately small sample, but the budget lesson is the point: sample size can hide an order-of-magnitude cost difference even when prediction quality is essentially unchanged. Guizzo *et al.* made the analogous point for mutation testing: using the number of mutants as a surrogate for execution cost can threaten empirical validity [6]. We ask whether the same surrogate-cost threat exists for sampling-based performance prediction.

The proxy failure can be stronger than a same-size cost gap. In Figure 2, a `pilotCost` sample with 256 configurations is both cheaper to acquire and slightly more accurate than a `diversity` sample with 128. Under a count budget the former is simply "larger" and therefore more expensive; under measured cost it is the cheaper and better choice. Our question is therefore not only whether n correlates with cost in aggregate, but whether n is a valid cost axis for the strategy choices a paper recommends.

This is not a blanket criticism of prior work. Sample size is practical, reproducible, and often correlated with actual cost. Our claim is narrower: *sample size can order sampling effort, but treating it as the cost axis is unsafe until the measured acquisition cost has been checked, and in many realistic sce-*

Sampling principle. A strategy selects n configurations from the option space; the measured runtime of those configurations is the acquisition cost, and their energy labels train the predictor.

$$\mathcal{C} \xrightarrow[\text{strategy } s]{n=16} S_{s,n}, \qquad TrainCost = \sum_{c \in S_{s,n}} \text{runtime}(c)$$

Excerpt of selected rows.

Strategy	enc.	comp.	method	lev.	cores	runtime
pilotCost	off	off	none	1	1	10.3k
pilotCost	off	off	none	2	3	10.4k
pilotCost	on	off	none	7	1	13.7k
diversity	on	on	zpaq	9	1	1.66M
diversity	off	on	zpaq	9	1	1.61M
diversity	off	on	lzma	9	4	288k

Rows are real selected configurations from the $n = 16$ samples. enc. and comp. are selected/deselected options; runtime is the cost paid before fitting the energy predictor.

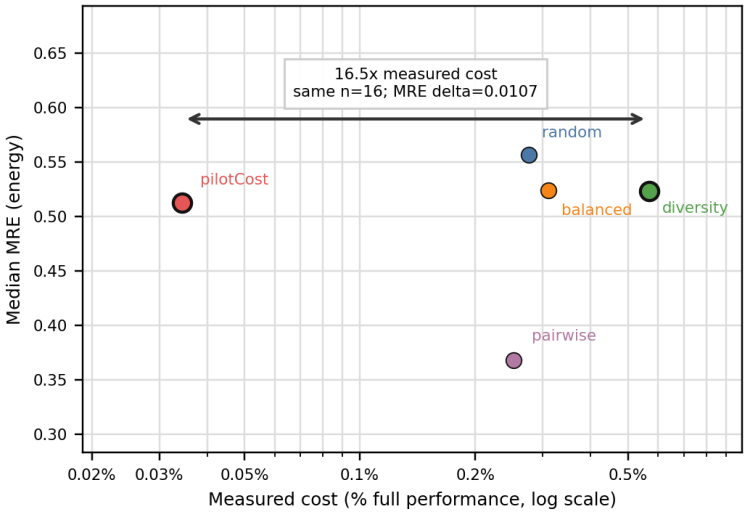


Fig. 1: Integrated running example on lrzip 0.631 compressing a 621 MB UIQ2 workload file (5,184 configurations, 12 option columns, runtime and energy measured over five repetitions). Left: the sampling principle and actual configuration excerpts show how two same-size samples can visit different option regions. Right: all strategies use $n = 16$, but measured performance cost separates nearly equal-error choices: pilot-cost and diversity differ by $16.5\times$ measured cost while MRE changes only from 51.3% to 52.4%.

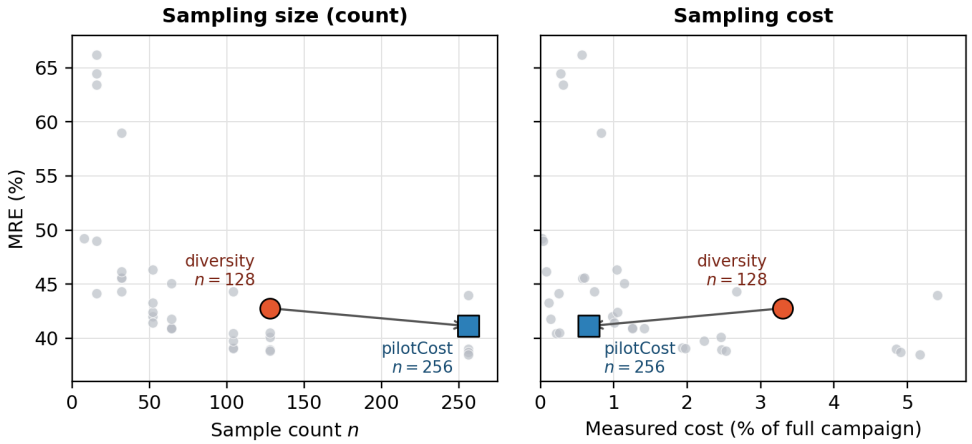


Fig. 2: A larger lrzip sample can be cheaper and more accurate under measured cost. The left panel uses sample count as the cost axis; the right panel uses measured acquisition cost for the same deployable strategy–budget combinations. The orange circle is `diversity` at $n = 128$ (MRE 42.77%, measured cost 3.297% of the full campaign). The blue square is `pilotCost` at $n = 256$ (MRE 41.11%, measured cost 0.658%). Grey points are the remaining deployable combinations in the same scenario.

narios that check changes the strategy recommendation or the set of efficient strategy–budget combinations. A statement such as "strategy A is cheaper than strategy B" is meaningful only after specifying whether "cheaper" means fewer configurations or less measured cost.

We test the size-as-cost abstraction by replacing count-based effort with measured acquisition cost[1]: the sum of the measured costs of the configurations a strategy selects. We study six complementary evidence families: a REPLICATION-ICSE19 revisit of Kaltenecker-style configurable-system traces, a REPLICATION-ICPE20 revisit of x264 over 17 inputs and two targets, a performance–energy corpus, a workload-impact corpus with 93 system–workload contexts, an LCBENCH external-validation corpus with 35 OpenML datasets and runtime learning curves, and seven Linux kernel releases with storage-bound cost. We ask three research questions:

1) to what extent sample size reflects measured acquisition cost;
2) whether replacing sample size by measured cost changes sampling-strategy conclusions; and
3) under which conditions sample size is a safe or unsafe cost proxy.

Agreement or disagreement across families is itself part of the result.

Contributions.

[1]Measured acquisition cost is the observed resource consumed to acquire a training label. For readability we use *measured cost* in the remainder.

- We derive the *Cost-Proxy Audit*: a lightweight, reusable, three-step check (plus a reporting checklist) that certifies whether sample size is a valid acquisition-cost budget for a given sampling study.
- We validate the audit across six evidence families—two replications of published configurable-system sampling settings and four extensions that vary cost dimension, workload, dataset family, and scale—showing it flags real failures: replacing count by measured cost changes $538/1{,}666$ top strategy–budget decisions (32%) and removes about half of the measured-cost-efficient choices from count-based Pareto fronts.
- We characterize the construct-validity mechanism the audit detects: heterogeneous per-configuration costs combined with strategy bias, so same-size samples spend very different budgets.
- We give the validity conditions that determine the audit's verdict: count is safe under tight costs, matching target and resource, and cost-blind strategies; it is unsafe under heavy-tailed costs, target–cost asymmetry, workload shifts, and diversity or active-learning samplers.

Section II recalls the sampling setting, Section III defines the measured cost model and decision tests, Section IV states the research questions and evidence families, Section V reports the findings in RQ order, Section VI interprets them, Section VII discusses threats, Section VIII places the study in context, and Section IX concludes.

II. Background

A configurable system exposes options whose valid combinations form a configuration space C. For a fixed workload and non-functional property, each configuration $c \in C$ has a measured value y_c such as runtime, output size, or energy. Sampling-based performance prediction trains a regression model $\hat{y}$ from a measured subset $S \subset C$ and evaluates prediction error, typically mean relative error ($MRE = (1/|T|)\sum_{c\in T} |y_c - \hat{y}_c|/|y_c|$; lower is better), on held-out configurations T [3]–[5], [7].

Sampling strategies.: The literature contains several domain-agnostic sampling families. Random sampling draws configurations uniformly when enumeration is available. Solver-based sampling asks SAT/SMT solvers for valid configurations; randomized solver-based sampling perturbs solver order to increase diversity [8]. t-wise sampling covers combinations of option values [9]–[11]. Distance-based and diversified distance-based sampling spread the sample in feature space [4]. When artifacts provide measured pools rather than feature-model handles, we use pool-based analogues: *balanced* keeps option frequencies close to the pool, *coverage* greedily matches marginal option coverage, *diversity* spreads by Hamming distance, *uncertainty* adds configurations with large predicted residuals, *pairwise* covers option-value pairs, and *pilot-cost* learns a small pilot cost model and then prefers configurations predicted inexpensive. Cheap and expensive oracles select the lowest- and highest-cost measured configurations; they are diagnostic bounds, not deployable samplers, and are excluded from all reported recommendations unless marked diagnostic.

How prior work counts cost.: Prior work has usually equated measurement effort with sample size. Sarkar *et al.* define a composite objective with training, testing, model, and error cost, but collapse acquisition effort to a term proportional to n under unit per-configuration cost [2]. Westermann *et al.* discuss the number of measurement points needed for a target error [12]; Guo *et al.* vary several count budgets [1]; Siegmund *et al.* report accuracy–count Pareto trade-offs [3]; and Kaltenecker *et al.* compare strategies at common sample sizes [4]. Siegmund *et al.* also report elapsed hours for some heuristics [13], but not as a test of whether n is a valid cost proxy. To our knowledge, the proxy itself has not been evaluated in this literature.

Context sensitivity.: Sampling conclusions already depend on workload and target property. Pereira *et al.* show that x264 strategy rankings change across 17 input videos and two properties [7]. We ask a different question: not whether prediction accuracy is context-sensitive, but whether the *validity of the cost proxy* is. The two can come apart—a workload can leave rankings stable while changing the measurement budget, or keep count and measured budgets close while changing the accuracy–cost frontier.

III. Cost Model and Decision Tests

For a run r of strategy s at training size n, let $S_{r,s,n} \subseteq C$ be the selected sample. The usual cost model is the count of measured configurations:

$$SizeCost(s, n) = n. \tag{1}$$

We compare it with measured training acquisition cost:

$$TrainCost(r, s, n) = \sum_{c\in S_{r,s,n}} cost(c). \tag{2}$$

Here $cost(c)$ is the measured resource consumed to acquire the label for configuration c. In most of our study it is wall-clock measurement time; in the performance–energy family it can be energy. The model does not require the cost dimension to equal the prediction target: one may predict output size while paying time, or predict energy while paying time. In the LINUX family the paid resource is not wall-clock time but the uncompressed `vmlinux` size, a storage-bound cost; we include it as a *resource-budget extension* to test whether the count-versus-cost threat also appears when the measured resource is a non-functional quantity other than time.

To compare systems with different absolute costs, we normalize by the cost of measuring the whole available configuration space:

$$FullCost = \sum_{c\in C} cost(c), \tag{3}$$

$$TrainCostPct = \frac{TrainCost}{FullCost}, \qquad SizePct = \frac{n}{|C|}. \tag{4}$$

The count proxy has good *budget agreement* (or *cost-proxy calibration*) when the fraction of configurations measured

matches the fraction of measured cost spent: a 10% sample should spend about 10% of the measured budget. This is not statistical calibration of a prediction model; it is only a scale check on the cost proxy. We summarize it with a budget ratio:

$$Ratio(r, s, n) = \frac{TrainCostPct(r, s, n)}{SizePct(s, n)}. \quad (5)$$

$Ratio = 1$ means a 10% sample consumes about 10% of the full cost; $Ratio > 1$ means the strategy selected costlier-than-average configurations and $Ratio < 1$ cheaper-than-average ones. This is a per-run ratio; one-number summaries report the median unless stated otherwise.

For decision-level comparisons we use two standard views. First, we rank strategy–budget combinations—each combination is one sampling strategy at one budget level—with a normalized objective

$$J_\lambda^{\text{size}} = \widetilde{MRE} + \lambda \widetilde{SizeCost}, \quad (6)$$

$$J_\lambda^{\text{meas}} = \widetilde{MRE} + \lambda \widetilde{TrainCost}, \quad (7)$$

where the cost weight $\lambda \in \{0.1, 0.25, 0.5, 1, 2, 4, 10\}$ spans accuracy-heavy to cost-heavy choices. The tildes denote min–max normalization of MRE and the cost term over the *deployable* strategy–budget combinations within the same system–input–target context, so error and cost are on the same $[0, 1]$ scale before they are added; conclusions are stable under rank normalization. Second, we compare Pareto fronts in the MRE–cost plane: a combination is efficient if no other has both lower error and lower cost. **Unless explicitly marked diagnostic, every top-choice and Pareto-front statistic is computed over deployable strategies only**; the cheap/expensive oracles only bound the achievable effect of cost awareness. The main threat we test is whether the recommended strategy or the efficient set changes when $SizeCost$ is replaced by $TrainCost$.

This formulation clarifies how the conventional fixed-n protocol fits our analysis. When all strategies are compared at the same sample size and the cost axis is sample count, the cost term is constant and the ranking collapses to MRE—the standard pure-accuracy reading of many sampling papers, which remains valid for that question. The extra assumption appears only when the same fixed-n comparison is read as a fixed-*budget* comparison. A "flip" then has two flavors. If the count-cost winner is more accurate but more expensive under measured cost, the result is a genuine trade-off the reader may still accept. If a different combination is no less accurate and no more expensive, the count-cost recommendation is *dominated*; Pareto fronts expose this case without fixing a particular λ. We count only training acquisition: validation measurements are identical across strategies, whereas a sampling strategy controls which training configurations it asks us to measure.

IV. Research Questions and Study Design

The study asks one question: when is sample size a good enough proxy for measured acquisition cost, and when does that shortcut change the substantive conclusion? We organize the evaluation around three research questions.

- **RQ1. To what extent does sample size reflect measured acquisition cost?** We test whether same-size samples spend comparable measured budgets, whether larger samples reliably cost more, and whether a fixed fraction of configurations corresponds to the same fraction of measured acquisition cost.
- **RQ2. Does replacing sample size by measured acquisition cost change sampling-strategy conclusions?** We compare the top accuracy–cost choices and the Pareto-efficient choice sets under count-based and measured cost.
- **RQ3. Under which conditions is sample size a safe or unsafe cost proxy?** We analyze how the answer depends on cost heterogeneity, strategy bias, target–cost asymmetry, workload/input, release, and dataset family.

Operational tests.: The tests are deliberately simple because the construct is simple. For RQ1, we measure *cost heterogeneity* with the coefficient of variation (CV, the unit-free ratio of standard deviation to mean) paired with p_{90}/p_{10}, the ratio between the 90th and 10th percentile costs ($p_{90}/p_{10} = 1.5$ means a costly but non-extreme configuration costs 50% more than a cheap one; a value near 47 means the upper tail is about $47\times$ the lower tail). We measure *budget calibration* with Spearman ρ between n and $TrainCost$ (does larger cost more?) and the stricter ratio $Ratio$ (Equation (5)). Fixed-n strategy-effect tests ask the operational question: when two strategies both choose, say, 64 configurations, do they visit systematically cheaper or costlier regions? For RQ2, a *top*-1 *flip* means the strategy–budget combination recommended by the same accuracy–cost objective changes after replacing count by measured cost, and *Pareto-front Jaccard* overlap measures the broader choice set (Jaccard $= 1$ means identical efficient sets). RQ3 interprets the same statistics across contexts. We report p-values where meaningful but always pair them with practical thresholds: CV≥ 0.20 or $p_{90}/p_{10} \geq 1.5$ for visible spread, $|\widetilde{Ratio} - 1| \geq 0.05$ for budget mismatch, and Jaccard < 1 for a changed efficient set. Full per-scope tests, p-values, and effect sizes are in the artifact.

A. Evidence Families

We keep six evidence families separate because they stress different validity conditions; pooling them would give a larger number but a weaker explanation of when the proxy is reliable. The two replication families keep the link with published configurable-system sampling studies: one revisits seven Kaltenecker-style systems, the other revisits x264 over 17 inputs and two measured properties. The four extension families then change one condition at a time: the cost dimension (performance versus energy), the workload or input, an external hyperparameter-optimization corpus, and a large Linux storage-cost setting.

The REPLICATION-ICSE19 family is the most conservative test. It reuses the published strategy panel—random, solver-based, randomized solver-based, t-wise, distance-based, and diversified distance-based sampling—and changes only the cost axis from sample count to measured performance

cost. If a count-based conclusion changes there, the change is not caused by inventing a new sampler or benchmark. The REPLICATION-ICPE20 family uses the same logic on x264, but adds context: the same 1,152 configurations are observed on 17 videos and two properties, so we can ask whether the proxy is safer when target and acquisition cost coincide than when they differ. The extension families broaden the construct: ENERGY asks whether the issue appears when performance and energy can each be target or cost; WORKLOAD keeps systems fixed but changes workloads, testing whether a conclusion is a property of the system or of the system–workload pair; LCBENCH moves outside configurable-software tables to real runtime learning curves; and LINUX stresses scale and strategy behavior, with acquisition cost the uncompressed kernel image size rather than time.

Table I gives the factual dataset scale; the result sections explain what these data imply for the three RQs.

B. Protocol and Artifact

For each available tuple of system, workload/input, target, strategy, training size, and run, we replay or generate the training sample, compute $TrainCost$, train the predictor on configuration options only, and evaluate MRE on held-out configurations. Stochastic strategies use repeated seeds when available, typically 100 runs. We exclude measured performance, energy, size, and runtime columns from the feature matrix to avoid label leakage, especially when target and acquisition cost are different measured quantities. All decision statistics are computed over deployable strategies; oracle-inclusive numbers are reported only as diagnostic bounds.

Artifact.: Our anonymized artifact contains the selected configurations, strategy identifiers, seeds, measured targets, and measured acquisition costs, together with the scripts that recompute $TrainCost$, $Ratio$, top-choice flips, and Pareto fronts, and instructions for reproducing every table and figure. The recorded fields—selected configurations, strategy/seed, measured target, and measured acquisition cost—are sufficient to recompute every ranking and front, so the count-versus-cost check can be replayed on the included data or applied to a new campaign.

V. Results

The empirical answer is not "sample size is useless." It is sharper: *sample size reliably orders effort, but it often hides the budget and can change decisions*. We report the families separately. REPLICATION-ICSE19 and REPLICATION-ICPE20 ask whether published count-based conclusions survive a measured-cost axis; ENERGY, WORKLOAD, LCBENCH, and LINUX test whether the phenomenon survives changes in cost dimension, workload, dataset family, and configuration-space scale.

A. RQ1: Sample Size Orders Cost, But Does Not Budget It

Sample size is a useful ordering signal but not a calibrated budget. In every family where we can test it, larger samples are positively rank-correlated with measured acquisition cost: Spearman correlation is high in the REPLICATION-ICSE19 revisit ($\rho = 0.879$–0.943), stays positive even in the hardest ENERGY cases ($\rho = 0.710$–0.996), and is near monotone across the seven Linux releases. Count-based plots are therefore not arbitrary, and n remains a useful common language for comparing campaigns.

The failure is budget calibration. At fixed n, strategies can visit very different cost regions. In the running example, five lrzip strategies all use $n = 16$, yet pilot-cost spends 0.034% of the full measured performance budget while diversity spends 0.567%—a $16.5\times$ gap for nearly identical MRE. Across the broader evidence, spread ranges from benign cases such as LLVM ($p_{90}/p_{10} = 1.21$) to heavy-tailed cases such as lrzip and BerkeleyDBC ($p_{90}/p_{10} \approx 47$), jump3r performance (CV 7.104), Linux storage cost (CV 0.99–1.43), and LCBENCH runtime (CV 0.595–2.247). The budget ratio tells the same story: a nominal 10% sample can spend close to 10%, but BerkeleyDBC has median $Ratio = 0.797$, ENERGY reaches $Ratio = 0.168$, workload contexts span 0.661–1.059, and LCBENCH spans 0.669–1.012.

The x264 revisit explains why aggregate calibration is not enough. Its time budgets look well calibrated when averaged over inputs, yet strategies are not interchangeable: Henard-style samples tend to be cheaper, t-wise samples costlier. A family can thus pass the coarse "does n order cost?" check and still fail the same-size, strategy-specific budget check that matters for recommendations. Linux illustrates the same split: random-like strategies stay close to the count budget, while diversity costs 1.24–$1.52\times$ its count share at small-budget levels and uncertainty can be cheaper or costlier depending on release and size. The three RQ1 checks can therefore disagree, and the same-size strategy-bias check is the one that predicts decision failures. Figure 2 shows the sharpest local case: under measured cost `pilotCost` at $n = 256$ spends 0.658% against `diversity`'s 3.297% at $n = 128$, so the larger sample is $5.0\times$ cheaper and slightly more accurate (MRE 41.11% vs. 42.77%).

Figures 3 and 4 make the RQ1 split concrete on the conservative REPLICATION-ICSE19 panel. Figure 3 shows that cost spread is itself a system property: per-configuration cost is nearly uniform on LLVM (CV 0.07, $p_{90}/p_{10} = 1.21$) but heavy-tailed on lrzip and BerkeleyDBC (CV above 1.4, $p_{90}/p_{10} \approx 47$). Figure 4 shows the consequence at a fixed budget: relative to random at the same n, solver-based and Henard-style samples spend as little as 17–28% of random's measured cost on BerkeleyDBC and lrzip, while t-wise and distance-based strategies sit near or above parity. Two studies that both report "n=64" on the same system can therefore differ several-fold in measured budget without disagreeing about sample size—the mechanism behind every same-size flip in RQ2.

TABLE I: Dataset characteristics. Configurations are measured configurations available per context unless noted otherwise; NFPs are the measured non-functional properties used as targets or acquisition costs.

Family	Scope	Configurations	Options/features	NFPs used	Inputs/workloads
REPLICATION-ICSE19	Seven configurable systems from Kaltenecker *et al.* [4].	432–68,640 valid configurations per system.	11–54 options.	One performance-like target per system; the same measured value is charged as cost.	One context per system.
REPLICATION-ICPE20	x264 ICPE 2020 [7].	1,152 valid configurations.	16 binary options.	Encoding time and output size, combined into four cost–target scenarios.	17 input videos.
ENERGY	Seven systems from the performance–energy corpus.	180–6,840 measured configurations per system.	System-specific option vectors from the source artifacts.	Performance and energy, combined into four scenarios per system.	One workload per selected system.
WORKLOAD	Nine systems from the workload-impact ICSE 2023corpus.	191–6,764 configurations per system; 25,258 measurements overall.	System-specific option vectors from the source artifacts.	Wall-clock time, or oriented throughput for H2.	6–13 workloads per system, 93 contexts total.
LCBENCH	35 OpenML datasets from LCBench.	2,000 hyperparameter configurations per dataset.	7 hyperparameters.	Validation accuracy as target, total runtime as cost.	35 dataset contexts.
LINUX	Seven TuxKConfig Linux releases.	19,952–92,562 valid configurations per release.	494–497 retained kernel options.	Uncompressed `vmlinux` size as target and storage-cost proxy.	Seven releases, v4.13–v5.08.

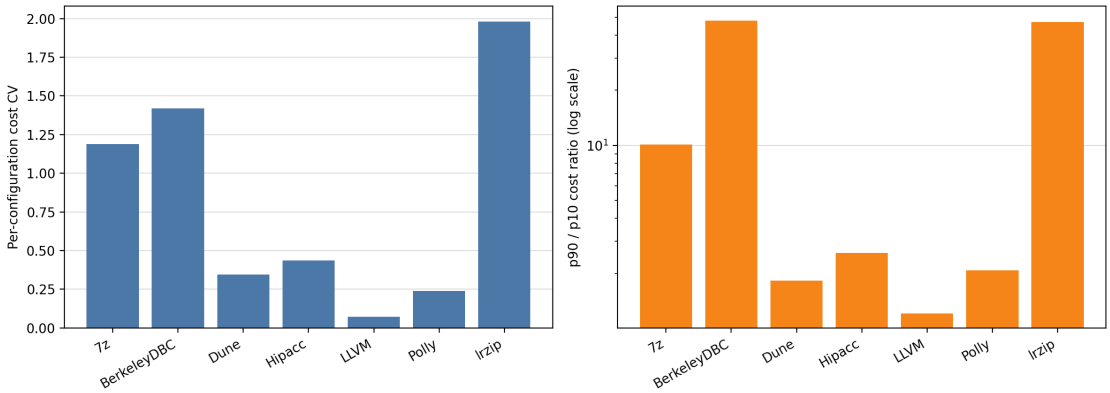


Fig. 3: Per-configuration cost spread on the REPLICATION-ICSE19 systems. Left: coefficient of variation; right: p_{90}/p_{10} ratio (log scale). LLVM and Polly are tight, while lrzip and BerkeleyDBC are heavy-tailed—the regime where a same-size sample can hide a large measured-cost gap.

Answer to RQ1

Sample size is a useful ordering variable, but not a calibrated budget. Across all families larger samples usually cost more, yet same-size samples can differ by orders of magnitude in measured cost, and a larger sample can be cheaper than a smaller one when the strategy selects cheaper configurations.

B. RQ2: Measured Cost Changes Strategy Assessment

Across all primary decision contexts, replacing sample count by measured acquisition cost changes the selected strategy–budget combination in $538/1{,}666$ *cases (*32%*).* Most changes are real accuracy–cost trade-offs rather than refutations: in $293/538$ changed cases (54%) the count winner is more accurate but also costlier to acquire. The stricter failure—where the count winner is *dominated* under measured cost, i.e. not on the measured-cost Pareto frontier—occurs in $57/538$ changed cases (11%). Beyond the top choice, count-cost frontiers miss about 52% of the strategy–budget combinations that are efficient under measured cost. The cost axis can therefore change both the headline recommendation and the efficient choice set, which is why RQ2 is about conclusions, not only accounting.

Here a *recommendation* is the strategy and budget level the objective would tell a practitioner to use for one system, input, target, and cost weight; a changed recommendation means the paper would name a different sampler or size once the measured budget is charged. A *choice set* is broader: the non-dominated combinations that remain plausible for some accuracy–cost preference. A stable top choice can still hide a changed choice set, which matters when users have different budgets or error tolerances.

Both effects appear across the families. In the REPLICATION-ICSE19 revisit, LLVM, Polly, Dune, and HIPAcc validate the top count-based recommendation, but lrzip changes in $5/7$ cost-weight settings and frontiers change on $5/7$ systems. The x264 revisit is stable in the symmetric time/time case, yet the time-cost/size-target scenario changes $21/119$ input–weight decisions with median Pareto Jaccard 0.375. Beyond the replications, top combinations change in $24/28$ ENERGY scenarios, $72/93$ WORKLOAD contexts, $33/35$ LCBENCH datasets, and every Linux release; Pareto fronts differ in all 28 ENERGY scenarios, $71/93$ workload contexts, $32/35$ LCBENCH datasets, and all seven Linux releases.

Figure 5 isolates a common source of confusion: the property we predict and the resource we pay to acquire a label need not be the same. For energy prediction the label is energy, but obtaining it requires running the configuration, so execution time is the natural acquisition cost. Holding that resource fixed and changing only the target flips the cost-model conclusion—

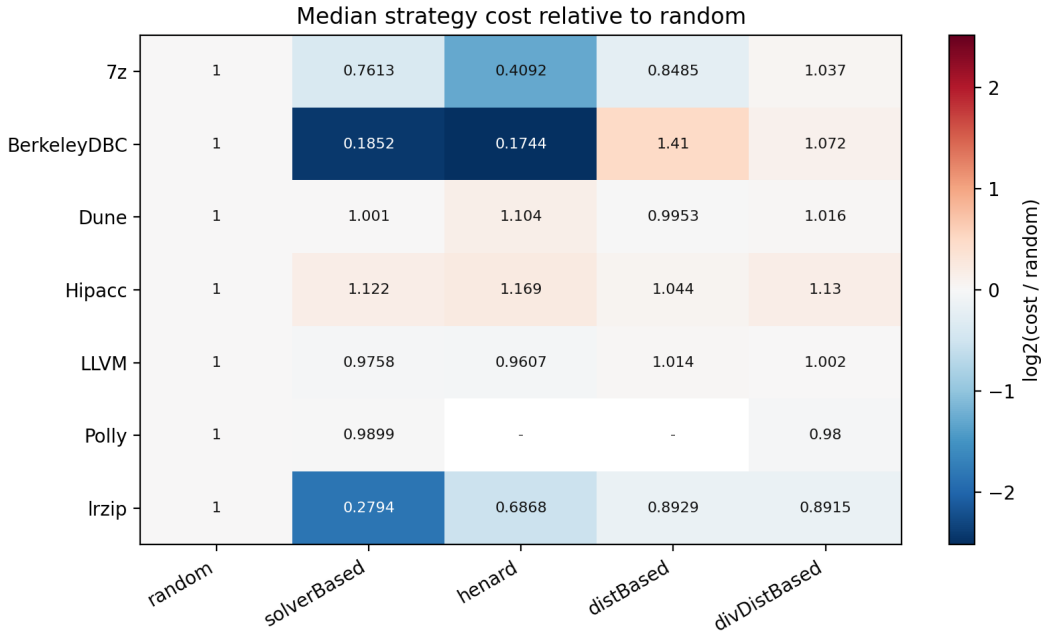


Fig. 4: Same-n strategy bias on the REPLICATION-ICSE19 systems: median measured cost of each strategy relative to random at the same sample size ($\log_2$ scale). Values far from parity mean the strategy systematically visits cheaper (blue) or costlier (red) configurations than its count implies; the effect is largest exactly on the heavy-tailed systems of Figure 3.

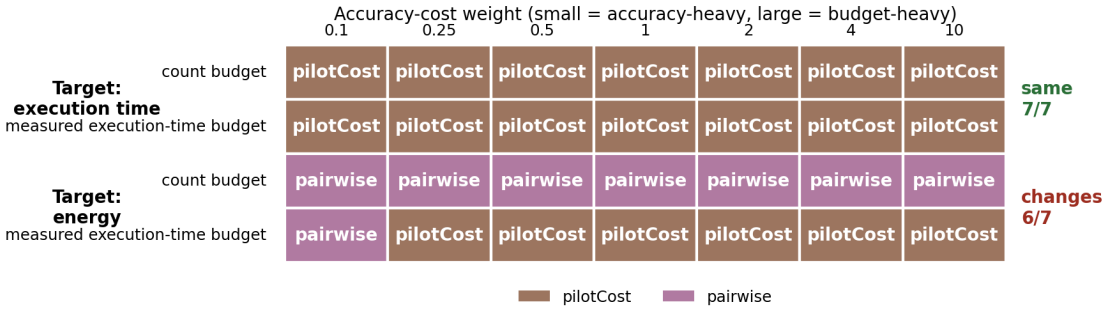


Fig. 5: Target–cost asymmetry in ENERGY on exastencils, over deployable strategies only. Both blocks charge the same acquisition resource, measured execution time. When the target is also execution time, count cost and measured cost recommend the same strategy for all seven accuracy–cost weights. When the target is energy—while acquisition still costs execution time—the recommendation changes for six of seven weights. This is why measured cost should be the resource paid to obtain labels, not automatically the property being predicted.

the time/time case validates the count recommendation, the time/energy case does not. The cost model is therefore part of the experimental question, not a synonym for the response variable.

The changed conclusion is not always "the count winner was worse." If the goal is pure MRE at a fixed sample size, the most accurate count-based strategy is the right answer; the study has intentionally held n constant and abstracted cost away. RQ2 does not penalize accuracy. It asks whether the accuracy improvement is still persuasive after charging the resource actually spent to obtain the labels. The strict failures are the cases where it is not: measured cost reveals another combination that is no less accurate and no more expensive, which the count axis had mispriced.

Answer to RQ2

Replacing sample size by measured cost changes both top recommendations and the efficient choice set. Across 1,666 context–weight decisions, the top strategy–budget combination changes in 538 cases (32%). Some are genuine accuracy–cost trade-offs; the strict failures are the 57 changed cases where the count-cost winner is dominated under measured cost.

C. RQ3: Validity Conditions

The proxy's safety is itself a property to measure. The count proxy is most defensible when per-configuration costs are tight, the target and acquisition resource coincide, and the strategy does not bias itself toward costly regions. It is most fragile when costs are heavy-tailed, target and cost differ, workload or dataset changes the cost distribution, or the strategy explicitly seeks diverse or uncertain regions.

The validated cases bound the claim. LLVM, Polly, Dune, HIPAcc, PostgreSQL, and x264 time/time show that count-based conclusions can survive when cost spread is low or strategies sample similar regions. The warning cases are different: 7Z and BerkeleyDBC have changed frontiers but few top flips, so sample size is insufficient for reporting the efficient choice set even if the headline winner survives. The changed-assessment cases combine cost spread with decision movement: lrzip flips in most cost weights, x264 changes most under time-cost/size-target asymmetry, LCBench changes on most datasets, and Linux changes on every release.

Four mechanisms recur, in increasing severity: a same-size choice can have a large measured-cost gap; two choices can have similar MRE but very different cost; a larger sample can be cheaper in measured time; and classic same-budget configurable-system strategies can separate once measured cost is charged. Each is a finite decision effect, not an estimate of a continuous Pareto curve: the object that matters is the set of concrete strategy–budget combinations a practitioner would actually compare after paying the measured budget, and count-based plots can hide it. The artifact includes a per-case gallery of these measured-cost decision plots.

On the REPLICATION-ICSE19 panel the split is visually separable: the systems that change a recommendation are the heavy-tailed ones in Figure 3 with off-parity columns in Figure 4, while the validated-transfer systems are tight in both. This suggests using the two RQ1 plots as a screening test—when cost spread is small and per-strategy ratios sit near parity, the measured-cost recomputation rarely moves the recommendation, so an author may keep the count budget with an explicit justification; when either condition fails, the recomputation is worth running before any cost-effectiveness claim is made.

Cross-RQ synthesis.: The families fall into three regimes. In the *validated-transfer* regime (LLVM, Polly, Dune, HIPAcc, PostgreSQL, x264 time/time), costs are tight or strategies cost-blind enough that replacing count by measured cost does not change the top recommendation. In the *warning* regime (7Z, BerkeleyDBC), the top recommendation mostly survives but the efficient choice set changes, so a reader comparing trade-offs would miss measured-cost-efficient alternatives. In the *changed-assessment* regime (lrzip, asymmetric x264, heavy-tailed performance–energy systems, many workload contexts, most LCBENCH datasets, and every Linux release), measured cost changes the recommendation itself. The scientific message is not that one sampler dominates everywhere; it is that the validity of the count proxy is an empirical property of the dataset, workload, cost dimension, and strategy panel.

Answer to RQ3

Sample size is safest as a cost proxy when per-configuration costs are tight, the target and bottleneck resource coincide, and strategies are cost-blind. It is unsafe when costs are heavy-tailed, target and acquisition cost differ, workload or dataset changes the cost distribution, or the strategy deliberately searches diverse or uncertain regions.

VI. Discussion

Prior work is not wrong; some claims are under-specified.: The results do not make prior sampling studies wrong; they show that a size-only cost claim is under-specified. A study may legitimately make a *fixed-size accuracy claim* (best predictor when every strategy gets the same n) or a *count-budget claim* (best strategy when sample count is the intended budget); the construct-validity gap appears only when either is read as a *measured-budget claim* about the resource actually paid for training labels. The fair retrospective question is therefore not whether a prior paper used the wrong sampler, but whether its sample-size budget was decision-equivalent to the measured resource—which is exactly what the audit below checks.

Cost-aware sampling is an opportunity, not this paper's contribution.: The diagnostic cheap/expensive oracles show what would be possible if acquisition cost were known in advance; they are not deployable. `pilotCost`—spend a small pilot budget, learn which regions are cheap, then avoid expensive configurations when accuracy does not justify them—is closer to a deployable idea and explains several high-contrast examples, but it is not available in every family and can fail when cheap regions are uninformative. We therefore stop short of proposing an optimal cost-aware sampler; our contribution is the prior step of deciding whether the count axis is valid for the study at hand.

Measured cost matters most when heterogeneity meets strategy bias.: A large cost CV alone is not the warning sign. BerkeleyDBC and 7Z show that frontiers can change even when top choices survive, while lrzip, brotli, jump3r, LCBench, and Linux show the stronger case, where spread combines with a panel that actively selects different cost regions. x264 adds target–cost asymmetry: a property can be cheap to predict but expensive to acquire, so the proxy can fail even when the aggregate budget ratio looks benign.

What the check does and does not settle.: The check settles whether a count budget is decision-equivalent to the measured resource for a given study; it does not rank samplers in the abstract or claim a universally best strategy. A passing audit licenses the count-based conclusion for that dataset, workload, target, and panel—not beyond them—which is why we report families separately rather than a single pooled effect. A failing audit does not condemn the original sampler either: it relocates the claim from a count budget to a measured one, where the same comparison may still favor the same strategy, only at a higher and now-visible cost.

Reading prior efficiency claims.: The audit also lets a reader triage a published cost-effectiveness claim without rerunning it. If the original artifact records per-configuration cost, a "$k\times$ fewer measurements" result can be recomputed in measured units directly—our decision tests are exactly that recomputation. If it does not, the claim should be read as a *count*-efficiency result, valid for what it measured, until the cost axis is supplied. This is not hypothetical: in our panel the same fixed-n comparison flips between count and measured cost on lrzip, the asymmetric x264 scenario, and every Linux

release, so a count-only efficiency margin is a weak guarantee precisely where acquisition costs are heterogeneous.

The Cost-Proxy Audit.: Our central practical artifact turns the preceding analysis into three steps any sampling study can run before making a cost-effectiveness claim. It is cheap—Steps 1–2 need only the measurement pool or a small pilot, and Step 3 reuses the rankings a study already computes—and it is conservative: when costs are homogeneous and per-strategy ratios stay near 1, it certifies that count is a defensible budget proxy, so it does not burden studies in the benign regime. When costs are heavy-tailed, target and cost differ, or strategies enter costly regions, it fails and tells the authors to restate the conclusion under measured cost. The audit does not require anyone to optimize acquisition cost; it only requires studies making a resource claim to validate the resource proxy they use.

The Cost-Proxy Audit

When: before any cost-effectiveness, efficiency, or speedup claim expressed in sample count.

1. Inspect cost spread. On the measurement pool or a small pilot, compute the per-configuration cost CV and p_{90}/p_{10}. Spread is material if CV ≥ 0.20 or $p_{90}/p_{10} \geq 1.5$.

2. Check per-strategy budget agreement. Report $Ratio = TrainCostPct/SizePct$ *by strategy* at the nominal budgets used; flag any strategy with $|\widetilde{Ratio} - 1| \geq 0.05$, since a global average hides a consistently cheap or expensive sampler.

3. Recompute the decision under measured cost. Re-rank the strategy–budget combinations and recompute the Pareto front with $TrainCost$. If the top choice and front are unchanged, the count proxy is validated for that setting; otherwise, state which conclusion is count-based and which is measured-cost based.

Report: sample size n; measured training cost $TrainCost$; per-strategy $Ratio$; and the measured-cost Pareto front whenever a cost-effectiveness claim is made.

VII. Threats to Validity

Cost construct.: We define acquisition cost as the sum of measured per-configuration costs in existing artifacts. This captures the benchmarked resource, not every production overhead such as orchestration, retries, or warm-up. The relative questions we ask–whether same-size samples choose cheaper or costlier configurations–are less sensitive to machine-specific absolute times.

Protocol heterogeneity.: The evidence families use different strategy panels, learners, workloads, and cost dimensions, so we report them separately rather than pooling one effect size. RQ1 is learner-independent; RQ2 and RQ3 depend on MRE and could move with another learner or error metric. We keep original learners for the two replication families and report learner-sensitivity artifacts for stronger learners and newly framed families.

Learner choice.: A better learner can improve MRE and can change the winning strategy–budget combination, so it is a threat to any claim about a specific top-ranked sampler. It does not, however, define acquisition cost: RQ1 is computed before fitting any model, from the selected configurations and their measured costs. The Linux HGBT sensitivity run is the key sanity check. It substantially improves accuracy, yet measured cost still changes rankings and Pareto fronts on every release. Better predictors can change which combination wins; they do not make sample size a sufficient cost model.

Decision-rule sensitivity.: RQ2's flips depend on the cost-weight grid λ and the min–max normalization. We mitigate this in three ways: we sweep λ over two orders of magnitude rather than fixing one operating point; we report Pareto-front changes, which are λ-free; and the direction of the conclusions is stable under rank normalization. A flip that survives all seven weights and the Pareto view is therefore not an artifact of a particular utility function. The residual risk is a reader with an extreme preference (pure accuracy or pure cost) at the edge of the grid; for them the fixed-n accuracy reading and the cheapest-sample reading remain available and unchanged.

External validity and multiple tests.: The systems cover compilers, encoders, databases, archivers, energy measurements, workloads, machine-learning runtime, and Linux releases, but they remain benchmark artifacts. Practitioners should transfer the check, not the exact thresholds: inspect cost spread and per-strategy budget ratios on the target campaign. Because we run many per-scope tests, main claims rely on repeated patterns across families; the supplementary artifact records the per-scope statistics, p-values, and effect sizes.

VIII. Related Work

Sampling configurable systems.: Sampling-based performance prediction has studied random, solver-based, coverage-based, distance-based, progressive, and transfer-learning strategies [1]–[5], [14]. Most papers evaluate effort as sample size. Sarkar *et al.*'s cost-efficient model is the closest configurable-system precedent [2], but its acquisition term is still proportional to the number of measured configurations. Our contribution is not a new sampler; it is a validity study of that effort axis—whether the count budget is decision-equivalent to the measured resource budget.

Counting effort versus measuring acquisition cost.: This is the gap we target. Across tuning, transfer, and prediction, "effort," "efficiency," and "speedup" are overwhelmingly expressed as counts—of measurements, evaluations, queries, or target-environment samples—because counts are simple to compare, reproduce, and reuse [15]–[20]. Counts are a sound way to *order* effort, but they are a *budget* only if one count unit stands for a fixed amount of acquisition cost; when it does not, a "cost-effective" result is an accuracy result wearing a cost label. The exposure is sharpest where one method is declared more resource-efficient under a fixed count budget—tuning and self-adaptation papers that report speedup as a ratio of measurement counts [15]–[18]; these are exactly the conclusions our decision tests can move (RQ2). Prediction papers that add a secondary data-efficiency claim in sample count [19], [20] are exposed more narrowly: their accuracy contributions stand, but the fewer-samples reading must be calibrated before it is a cost claim. We do not revisit these

algorithms; we test the assumption beneath their efficiency axis.

Surrogate and heterogeneous costs.: Replacing a real resource with an easy-to-count surrogate is a recurring validity concern: when items differ in cost, the count and the resource can diverge. This surrogate-cost threat is documented in mutation testing, where the number of mutants can mislead relative to execution time [6]; we ask the analogous question for configuration sampling. The mechanism is specific to configurable systems: sample size is usually correlated with measured cost, yet strategy-specific budget agreement and decision-level frontiers can still change when some configurations are slow, energy-intensive, storage-heavy, or workload-dependent.

Calibrated counts are the right first step.: Part of this literature already moves in the direction we advocate, and we credit it. Multi-objectivizing tuning and its aspiration-guided successor start from a measured wall-clock budget on dedicated hardware and convert it into a per-system number of unique measurements used as the stopping criterion [21], [22]. These are complementary precedents that ground counts in a measured budget—exactly our recommendation. Our results push one step further: even a case-level calibration can drift from per-configuration acquisition cost when measurement costs are heterogeneous within a system or biased by a strategy, which is the regime where the count-versus-cost decision flips. Pereira *et al.* likewise show context sensitivity of rankings [7], and recent tuning-analysis work argues workload effects are system-dependent [23]; we use these context dimensions to test cost-proxy validity.

Active learning, optimization, and learners.: Cost-aware sampling can be cast as active learning or multi-objective optimization; both are candidate solutions, not competitors. Each inherits the proxy question through its cost term and is only as good as that term is calibrated, so we keep standard decision forms but evaluate them once with sample size and once with measured cost, following Li *et al.*'s argument that Pareto-based SBSE should match indicators to the decision question [3], [24]. Configuration-performance learners range from linear models and CART-style trees to random forests and boosted trees [4], [25]–[29]; we keep original learners for the two replication families and use stronger sklearn baselines as robustness checks, separating the modeling question from our cost question. A distinct strand measures the cost of *building* a model—encoding-scheme training time [30], drift-adaptation overhead [31], and the DCPL survey [32]; that is a complementary axis to the label-acquisition cost we study.

IX. Conclusion

Sampling cost matters. For configurable-system performance prediction, the number of measured configurations is a useful summary, but it is not a resource budget unless it has been checked against measured acquisition cost. Across the systems we studied, n almost always orders measured cost, yet it can misstate the budget, hide strategy-specific bias, and change which strategy or strategy–budget combination should be recommended.

A post-hoc decision audit makes the risk concrete. Across $1{,}666$ context–weight decisions, the top strategy–budget combination changes in 538 cases (32%) when measured cost replaces count cost. In $293/538$ changed cases (54%) the count winner is more accurate but costlier—a real trade-off; in the stricter $57/538$ cases (11%) it is dominated under measured cost. Count-based frontiers also miss about 52% of the Pareto-efficient combinations under measured cost.

The result is a construct-validity warning, not a blanket rejection of prior work. Count-based effort is defensible for homogeneous systems and cost-blind samplers, and our data identifies several such cases; it is unsafe for heavy-tailed costs, target–cost asymmetry, workload-sensitive cost distributions, dataset shifts, and diversity or active-learning strategies that drift toward costly regions. These conditions are common, measurable, and severe enough to change winners and Pareto fronts. The new LCBENCH family shows the warning is not confined to configurable-software corpora: the same count-versus-cost gap survives on runtime learning-curve benchmarks. The burden is therefore on future studies to show that their count budget is decision-equivalent to the measured resource, or to state the conclusion under measured cost. The fix is the lightweight Cost-Proxy Audit—inspect cost spread, check per-strategy budget agreement, and recompute the decision under measured cost—which turns "effort" from an unspoken convention into a verifiable part of the experimental design.

Acknowledgments

Anonymized for review.

References

[1] J. Guo, K. Czarnecki, S. Apel, N. Siegmund, and A. Wasowski, "Variability-aware performance prediction: A statistical learning approach," in *Automated Software Engineering (ASE)*. IEEE, 2013, pp. 301–311.

[2] A. Sarkar, J. Guo, N. Siegmund, S. Apel, and K. Czarnecki, "Cost-efficient sampling for performance prediction of configurable systems (t)," in *IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 2015, pp. 342–352.

[3] N. Siegmund, A. Grebhahn, S. Apel, and C. Kastner, "Performance-influence models for highly configurable systems," in *10th Joint Meeting on Foundations of Software Engineering*, ser. ESEC/FSE, 2015, pp. 284–294.

[4] C. Kaltenecker, A. Grebhahn, N. Siegmund, J. Guo, and S. Apel, "Distance-based sampling of software configuration spaces," in *Proceedings of the International Conference on Software Engineering (ICSE)*, 2019.

[5] J. A. Pereira, H. Martin, M. Acher, J.-M. Jézéquel, G. Botterweck, and A. Ventresque, "Learning software configuration spaces: A systematic literature review," 2019.

[6] G. Guizzo, F. Sarro, and M. Harman, "Cost measures matter for mutation testing study validity," in *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, ser. ESEC/FSE 2020. New York, NY, USA: Association for Computing Machinery, 2020, p. 1127–1139. [Online]. Available: https://doi.org/10.1145/3368089.3409742

[7] J. Alves Pereira, M. Acher, H. Martin, and J.-M. Jézéquel, "Sampling effect on performance prediction of configurable systems: A case study," in *Proceedings of the ACM/SPEC International Conference on Performance Engineering*, ser. ICPE '20. New York, NY, USA: Association for Computing Machinery, 2020, p. 277–288. [Online]. Available: https://doi.org/10.1145/3358960.3379137

[8] C. Henard, M. Papadakis, G. Perrouin, J. Klein, P. Heymans, and Y. L. Traon, "Bypassing the combinatorial explosion: Using similarity to generate and prioritize t-wise test configurations for software product lines," *IEEE Trans. Software Eng.*, 2014.

[9] D. Kuhn, D. Wallace, and A. Gallo, "Software fault interactions and implications for software testing," *IEEE Transactions on Software Engineering*, vol. 30, no. 6, pp. 418–421, jun 2004.

[10] M. B. Cohen, M. B. Dwyer, and Shi, Jiangfan, "Constructing Interaction Test Suites for Highly-Configurable Systems in the Presence of Constraints: A Greedy Approach," *IEEE TSE*, vol. 34, no. 5, pp. 633–650, 2008.

[11] M. F. Johansen, Ø. Haugen, and F. Fleurey, "An Algorithm for Generating t-Wise Covering Arrays from Large Feature Models," in *Proceedings of the 16th International Software Product Line Conference on - SPLC '12 -volume 1*, vol. 1. ACM, 2012, p. 46.

[12] D. Westermann, J. Happe, R. Krebs, and R. Farahbod, "Automated inference of goal-oriented performance prediction functions," in *IEEE/ACM International Conference on Automated Software Engineering (ASE)*. ACM, 2012, pp. 190–199.

[13] N. Siegmund, S. S. Kolesnikov, C. Kästner, S. Apel, D. S. Batory, M. Rosenmüller, and G. Saake, "Predicting performance via automated feature-interaction detection," in *International Conference on Software Engineering (ICSE)*, 2012, pp. 167–177.

[14] V. Nair, T. Menzies, N. Siegmund, and S. Apel, "Faster discovery of faster system configurations with spectral learning," *Automated Software Engineering*, pp. 1–31, 2018.

[15] V. Nair, Z. Yu, T. Menzies, N. Siegmund, and S. Apel, "Finding faster configurations using FLASH," *IEEE Transactions on Software Engineering*, 2020. [Online]. Available: https://doi.org/10.1109/TSE.2018.2870895

[16] P. Chen, T. Chen, and M. Li, "Mmo: Meta multi-objectivization for software configuration tuning," *IEEE Transactions on Software Engineering*, 2024, preprint at arXiv:2112.07303. [Online]. Available: https://doi.org/10.1109/TSE.2024.3388910

[17] T. Chen and M. Li, "Adapting multi-objectivized software configuration tuning," in *Proceedings of the ACM International Conference on the Foundations of Software Engineering*, 2024. [Online]. Available: https://doi.org/10.1145/3643751

[18] Y. Ye, T. Chen, and M. Li, "Distilled lifelong self-adaptation for configurable systems," *arXiv preprint arXiv:2501.00840*, 2025, accepted at ICSE 2025. [Online]. Available: https://arxiv.org/abs/2501.00840

[19] J. Gong and T. Chen, "Predicting software performance with divide-and-learn," in *Proceedings of the ACM International Conference on the Foundations of Software Engineering*, 2023. [Online]. Available: https://doi.org/10.1145/3611643.3616334

[20] ——, "Predicting configuration performance in multiple environments with sequential meta-learning," in *Proceedings of the ACM International Conference on the Foundations of Software Engineering*, 2024, preprint at arXiv:2402.03183. [Online]. Available: https://doi.org/10.1145/3643743

[21] T. Chen and M. Li, "Multi-objectivizing software configuration tuning (for a single performance concern)," in *Proceedings of the 29th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 2021. [Online]. Available: https://doi.org/10.1145/3468264.3468555

[22] ——, "Do performance aspirations matter for guiding software configuration tuning?" *ACM Transactions on Software Engineering and Methodology*, 2023. [Online]. Available: https://doi.org/10.1145/3571853

[23] Y. Ye, H. Liang, C. Jiang, M. Li, and T. Chen, "Revealing domain-spatiality patterns for configuration tuning: Domain knowledge meets fitness landscapes," *ACM Transactions on Software Engineering and Methodology*, 2026. [Online]. Available: https://doi.org/10.1145/3803859

[24] M. Li, T. Chen, and X. Yao, "How to evaluate solutions in pareto-based search-based software engineering? a critical review and methodological guidance," *IEEE Transactions on Software Engineering*, 2020. [Online]. Available: https://doi.org/10.1109/TSE.2020.3036108

[25] L. Breiman, "Random forests," *Machine learning*, vol. 45, no. 1, pp. 5–32, 2001.

[26] P. Valov, J. Guo, and K. Czarnecki, "Empirical comparison of regression methods for variability-aware performance prediction," in *SPLC'15*, 2015.

[27] B. Amand, M. Cordy, P. Heymans, M. Acher, P. Temple, and J.-M. Jézéquel, "Towards learning-aided configuration in 3d printing: Feasibility study and application to defect prediction," in *Proceedings of the 13th International Workshop on Variability Modelling of Software-Intensive Systems*. ACM, 2019, p. 7.

[28] L. Bao, X. Liu, Z. Xu, and B. Fang, "Autoconfig: Automatic configuration tuning for distributed message systems," in *IEEE/ACM International Conference on Automated Software Engineering (ASE)*. ACM, 2018, pp. 29–40.

[29] L. Buitinck, G. Louppe, M. Blondel, F. Pedregosa, A. Mueller, O. Grisel, V. Niculae, P. Prettenhofer, A. Gramfort, J. Grobler, R. Layton, J. VanderPlas, A. Joly, B. Holt, and G. Varoquaux, "API design for machine learning software: experiences from the scikit-learn project," in *ECML PKDD Workshop: Languages for Data Mining and Machine Learning*, 2013, pp. 108–122.

[30] J. Gong and T. Chen, "Does configuration encoding matter in learning software performance? an empirical study on encoding schemes," in *Proceedings of the 19th International Conference on Mining Software Repositories*, 2022. [Online]. Available: https://doi.org/10.1145/3524842.3528431

[31] Z. Xiang, J. Gong, and T. Chen, "Dually hierarchical drift adaptation for online configuration performance learning," in *Proceedings of the International Conference on Software Engineering*, 2026. [Online]. Available: https://doi.org/10.1145/3744916.3764542

[32] J. Gong and T. Chen, "Deep configuration performance learning: A systematic survey and taxonomy," *ACM Transactions on Software Engineering and Methodology*, 2024, preprint at arXiv:2403.03322. [Online]. Available: https://doi.org/10.1145/3702986